

BottleNeck

Automatiser le pipeline data nettoyage, jointure, CA et classement des vins

Kestra • Python • DuckDB

Daniel — Data Engineer

Le contexte de la mission

BottleNeck, marchand de vin prestigieux, vient de m'embaucher comme Data Engineer.

- **Laurent** Manager : a lancé la mission
- **Stéphane** Data Analyst : a fait l'analyse manuelle de départ
- **Laure** Clients entreprises & vins premium
- **Maria** Particuliers & vins ordinaires

Ma mission

Automatiser la chaîne de traitement que Stéphane a construite à la main, pour que Laure et Maria reçoivent leurs rapports chaque mois sans intervention manuelle.

Le besoin métier

Rapport CA

Chiffre d’affaires par produit + total, chaque mois

Vins classés

Extraction séparée des vins premium et ordinaires

Automatique

Exécution planifiée le 15 du mois à 9h, sans intervention

Objectif : que Kestra fasse tourner tout ça automatiquement, avec des garde-fous en cas de problème.

La démarche de Stéphane (le point de départ)

1

Nettoyage

Suppression des valeurs manquantes et doublons, sur ERP et CMS

→

2

Jointure

Réconciliation via le fichier liaison.xlsx

→

3

Calcul du CA

Par bouteille, puis chiffre d'affaires total

→

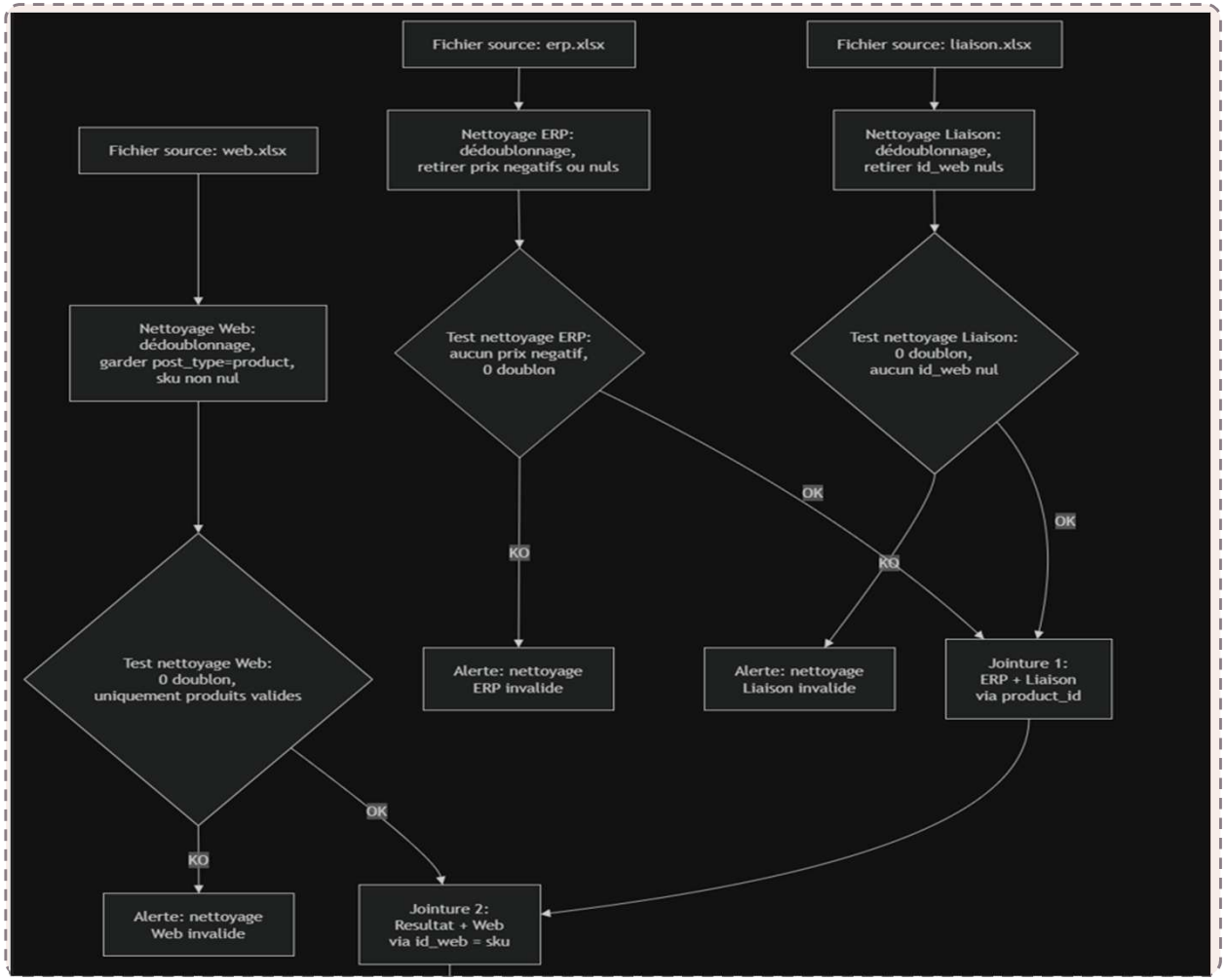
4

Classement des vins

Méthode du z-score sur le prix

Mon rôle : industrialiser ces 4 étapes avec Kestra, sans changer la logique métier validée en COPIL.

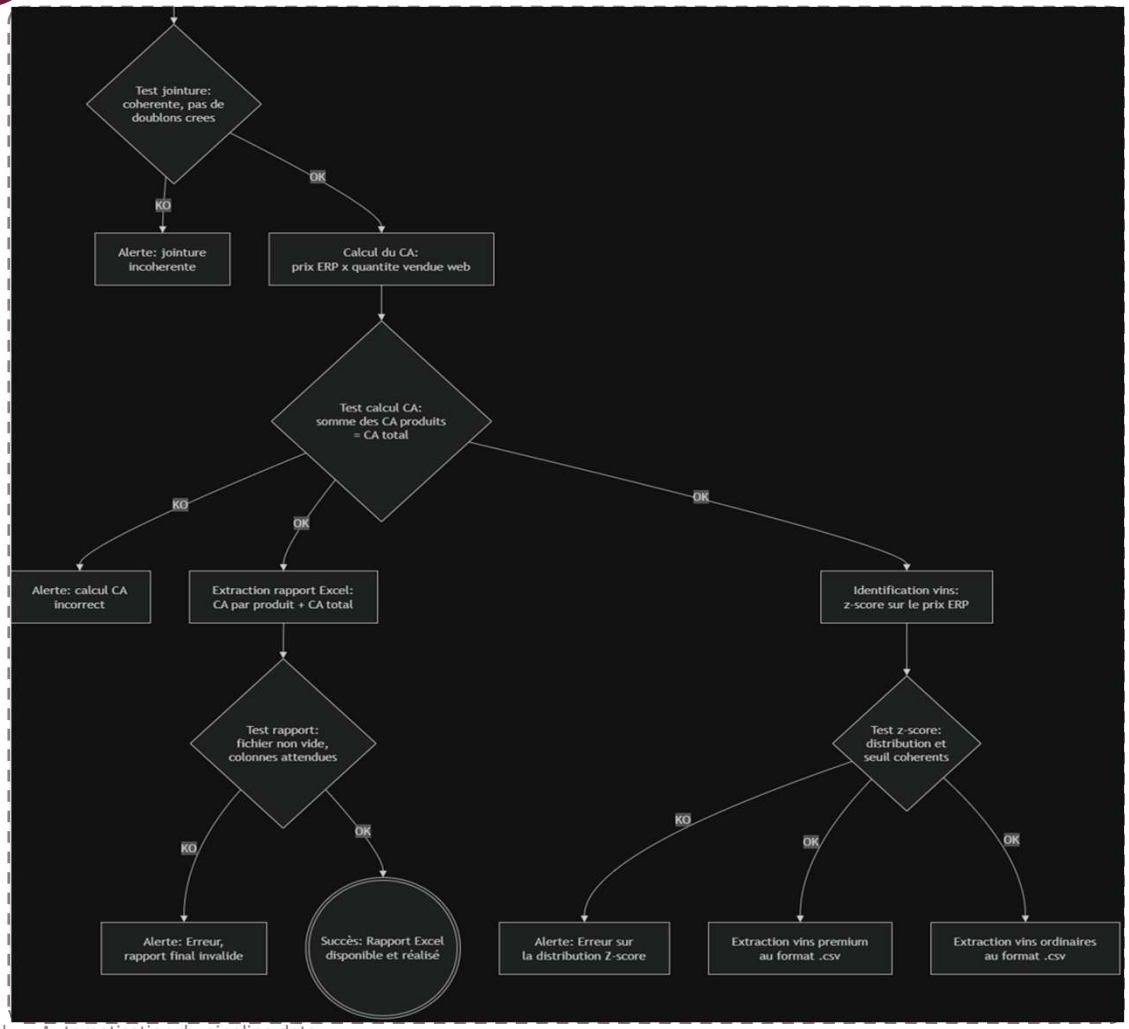
Étape 1 — Le logigramme (data lineage)



Ce que montre le schéma

- 3 nettoyages en parallèle (ERP, Web, Liaison)
- 2 jointures successives
- Calcul du chiffre d'affaires
- 2 branches finales : rapport Excel + classement des vins
- Un test après chaque tâche, avec une alerte dédiée en cas d'échec

Étape 1 — Le logigramme (data lineage)



Ce que montre le schéma

- 3 nettoyages en parallèle (ERP, Web, Liaison)
- 2 jointures successives
- Calcul du chiffre d'affaires
- 2 branches finales : rapport Excel + classement des vins
- Un test après chaque tâche, avec une alerte dédiée en cas d'échec

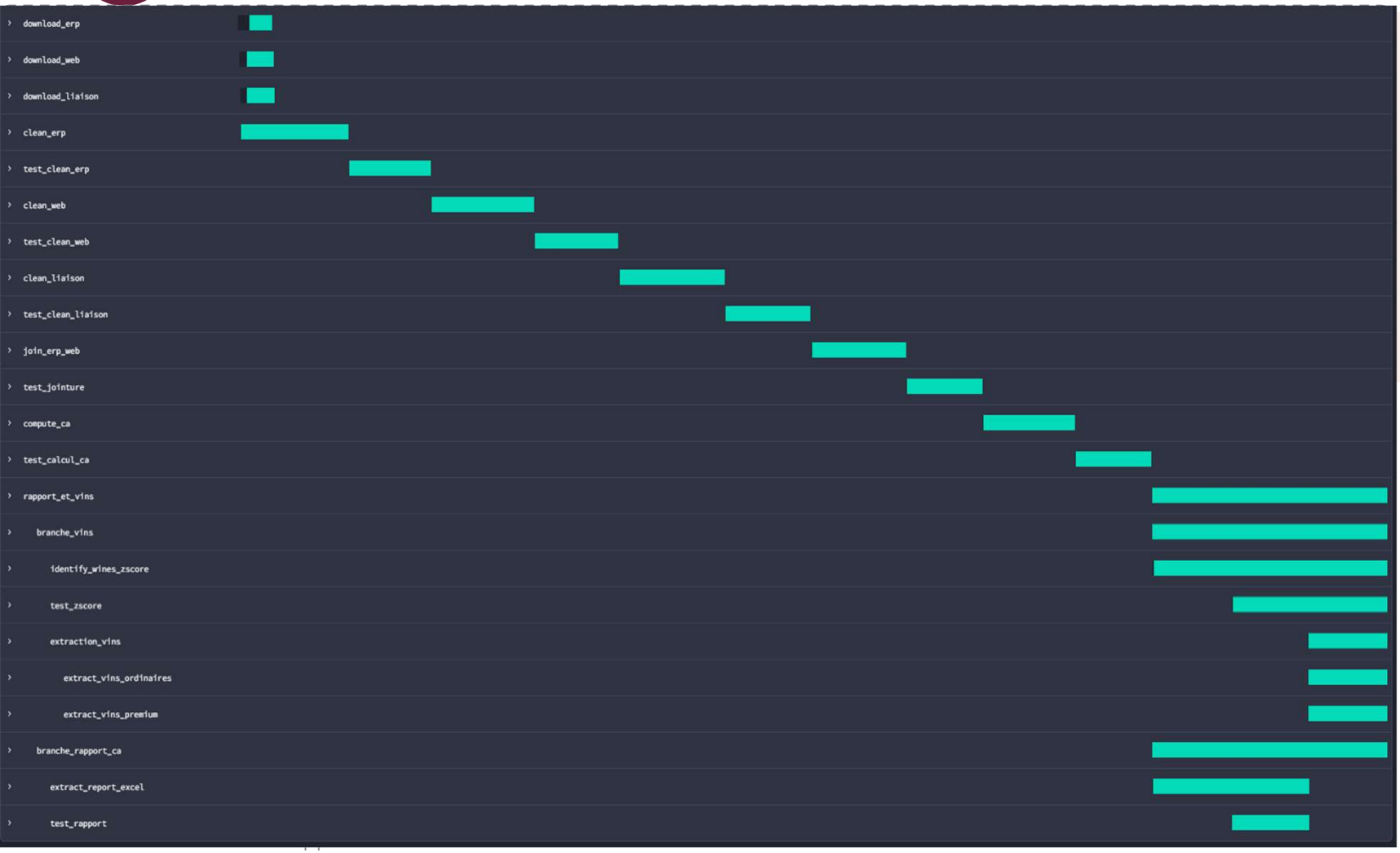
Kestra, en une phrase

Kestra orchestre — il ne transforme jamais lui-même les données.

C’est un chef d’orchestre : il décide quelle tâche joue, quand, et dans quel ordre. Ce sont les musiciens — nos scripts Python et nos requêtes DuckDB — qui font le vrai travail sur les données.

C’est pour ça que le point de vigilance officiel le rappelle : Kestra est un outil d’orchestration, pas de traitement de données.

Étape 2 — La topologie du workflow Kestra



15 tâches, 3 étages

- Nettoyages (Python + DuckDB embarqué)
- Jointures + calcul du CA
- Deux branches Parallel : rapport Excel // classement des vins
- Chaque tâche nominale a sa tâche de test juste après

Choix techniques & justifications

Python + DuckDB embarqué

Contourne les limites du plugin JDBC natif de Kestra pour DuckDB, tout en gardant du vrai SQL dans le script

Parallel / Sequential

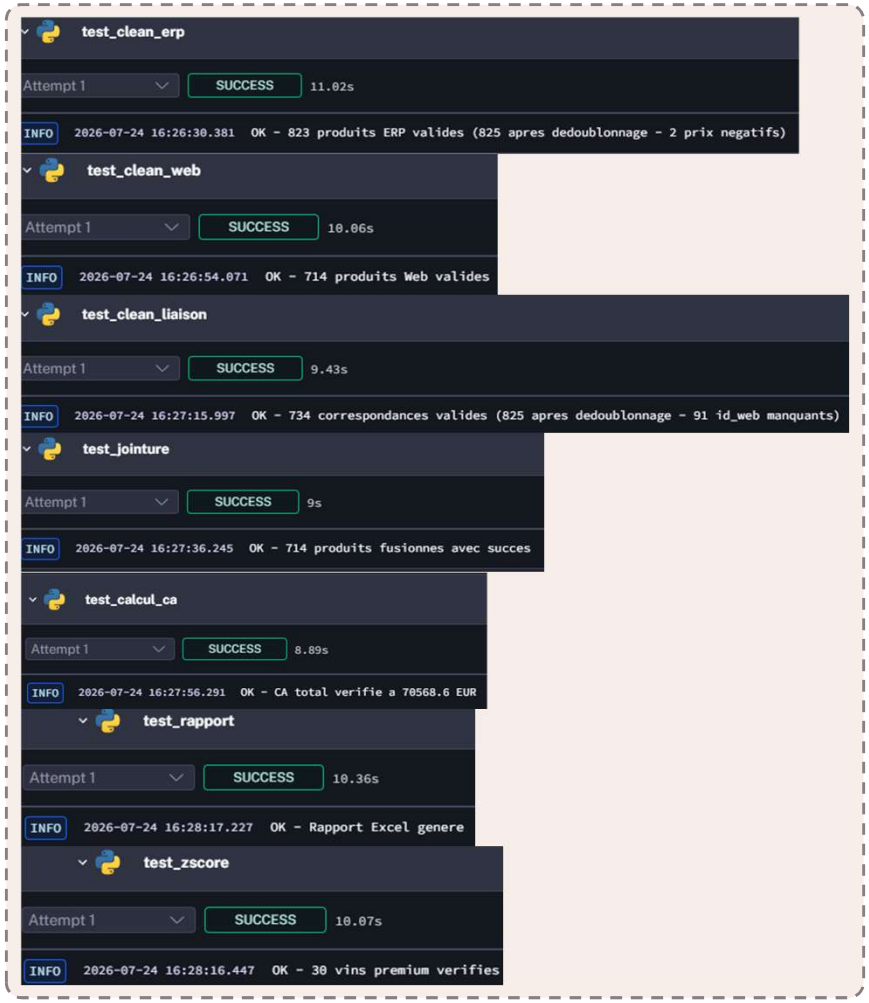
Le rapport Excel et le classement des vins ne dépendent pas l'un de l'autre → exécution simultanée, plus rapide

retry + errors

Si DuckDB ou une dépendance est momentanément indisponible, la tâche retente automatiquement avant d'alerter

Étape 3 — Les tests intégrés

- Absence de doublons (product_id, sku)
- Absence de valeurs manquantes
- Cohérence des jointures (taux de correspondance)
- Cohérence du chiffre d'affaires (somme = total)
- Test du z-score sur le prix des vins

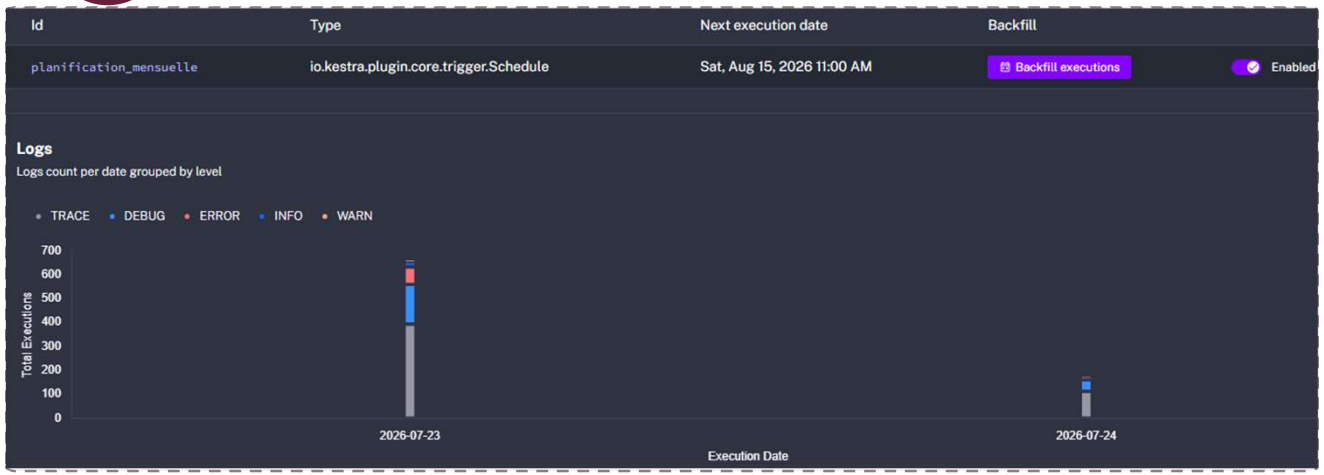


Bonus — Nos résultats face à ceux de Stéphane

Indicateur	Stéphane	Notre pipeline	Pourquoi
ERP dédupliqué	825	823	2 prix négatifs filtrés (règle qualité)
Liaison dédupliqué	825	734	91 produits sans présence en ligne retirés
Web dédupliqué	714	714	Match parfait
Fichier fusionné	714	714	Match parfait
CA total	70 568,60 €	70 568,60 €	Match parfait, calcul identique
Vins premium	30	30	Match parfait (z-score sur le fusionné)

Chaque écart s'explique par une règle de qualité de donnée assumée — jamais par une erreur de logique.

La planification automatique



Le 15 de chaque mois, à 9h

```
0 9 15 * *
```

minute · heure · jour du mois · mois · jour de semaine

Aucune intervention manuelle : le trigger Schedule de Kestra déclenche tout le workflow, sans qu'un humain clique sur Execute.

Robustesse face aux dysfonctionnements

3 tentatives automatiques

Chaque tâche DuckDB/Python retente jusqu'à 3 fois avant d'abandonner (utile si un service tiers est momentanément indisponible)

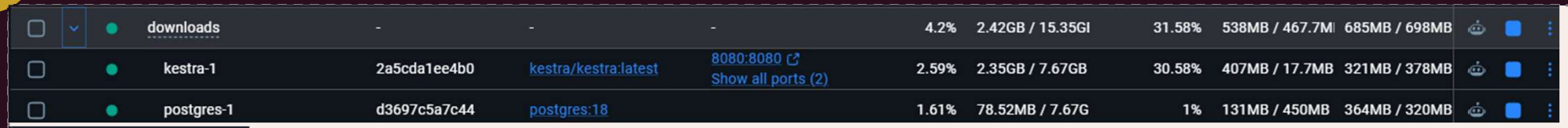
Alerte en cas d'échec définitif

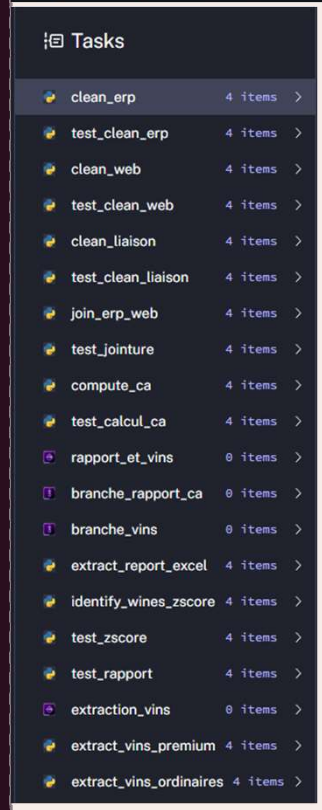
Un bloc "errors" au niveau du flow prévient l'équipe si tout échoue malgré les tentatives

Vigilance sur les nouvelles données

Un changement de structure dans les fichiers sources est détecté par les tests, avant de fausser un rapport

Démonstration en direct





Place à la pratique : exécution du pipeline en direct

Lancement de Kestra, suivi des tâches et extraction des livrables en temps réel.

Limites & pistes d’amélioration

Valeurs de test figées

Les tests comparent à des chiffres exacts (823, 70 568,60 €...) valables sur ce jeu de référence — à remplacer par des bornes de cohérence pour un usage mensuel réel

Alerte simplifiée

Le bloc d’erreur actuel journalise seulement — une vraie notification Slack ou email vers Laurent serait la suite logique



Merci — questions ?

- Un pipeline fidèle à la démarche validée de Stéphane, entièrement automatisé
- Un test après chaque tâche, et des résultats qui collent aux chiffres de référence
- Planifié, résilient, documenté — prêt pour la production